

T1 de Computação Gráfica

Visualização 2D do Cultural Dataset

Germano Bruscato Corrêa

¹Pontifícia Universidade Católica do Rio Grande do Sul - Escola Politécnica
Av. Ipiranga, 6681 - Partenon, 90619-900 - Porto Alegre, RS, Brasil

germano.bruscato@edu.pucrs.br

Resumo. *Este artigo apresenta o trabalho desenvolvido como avaliação da disciplina de Computação Gráfica. O trabalho consistiu em aplicar algoritmos de visualização e animação 2D com base nos dados de arquivos um dataset que contém trajetórias de entidades em um determinado plano. Essas entidades são desenhadas no plano e animadas com base em suas trajetórias definidas no arquivo mencionado. Uma entidade extra, denominada como avatar, é controlável pelo usuário com teclas direcionais do teclado. A animação das entidades não controláveis é suavizada utilizando interpolação linear, e a colisão entre quaisquer entidades é detectada em cada frame pelo método Axis Aligned Bounding-Box. A cada colisão detectada, a cor do par de entidades envolvidas na colisão é alterado para uma cor aleatória. Por meio desta avaliação, foi possível aplicar conceitos de visualização, animação e pipeline 2D, bem como técnicas diferentes para animar, suavizar e detectar as colisões.*

1. Introdução e Objetivo

Este trabalho faz parte da avaliação da disciplina de Computação Gráfica, cujo objetivo é utilizar algoritmos de visualização 2D para desenhar e animar dados em função do tempo. Um vídeo também foi confeccionado como parte da avaliação, no link: <https://youtu.be/l4yjAnwNw6c>

1.1. Objetivos do Projeto

Dentre os objetivos e requisitos, o trabalho deve:

1. Utilizar *OpenGL*.
2. Utilizar dados dos arquivos `Path_D.txt` do *Cultural Dataset*.
3. Visualizar e animar pelo menos 6 entidades com base na trajetória representada nestes arquivos.
4. Implementar algum processamento dos dados, com visualização.
5. Implementar alguma interação, via mouse ou teclado, para mover pelo menos uma entidade.
6. Apresentar o *pipeline 2D*.

2. Recursos do OpenGL Utilizados

O trabalho foi escrito em *C++* e utilizou o *OpenGL* para desenhar os objetos, aplicar transformações, colorir, e ainda definir a *viewport*; e a biblioteca *GLUT*, para organizar os *callbacks* de eventos, o *loop* principal de execução e o gerenciamento da janela de visualização.

2.1. Recursos Principais

- **Configuração de Viewport e Projeção:** Utilização de `gluOrtho2D` para mapear as coordenadas de recorte do mundo, definidas pelos limites dos dados de entrada, para a *viewport* da janela.
- **Transformações Geométricas:** Uso de `glPushMatrix` e `glTranslatef` para posicionar e orientar cada entidade no mundo 2D.

- **Desenho de Primitivas:** Entidades são representadas como quadrados desenhados com `glBegin(GL_QUADS)`, e seus vértices definidos através de `glVertex2f`. As primitivas são coloridas dinamicamente com `glColor3f`.
- **Exibição:** A função `glutDisplayFunc` gerencia a renderização de um *frame*, e `glutIdleFunc` para o *loop* de animação. Ainda usou-se do modo de exibição `GLUT_DOUBLE`, que permite desenhar um *frame* enquanto outro está sendo exibido, e da função e `glutSwapBuffers` no *callback* de `glutDisplayFunc` para realizar a troca entre esses dois *frames*.
- **Interação com Teclado:** uso das funções `glutKeyboardFunc` e `glutSpecialFunc` para mapear ações do teclado com ações no programa.

3. Pipeline

O *pipeline* do projeto segue as etapas convencionais de computação gráfica através de *OpenGL* e *GLUT*, com a adição das implementações específicas deste trabalho. Pela natureza orientada a eventos do *OpenGL*, é difícil definir em que seções do código deste trabalho são feitas cada etapa do *pipeline 2D*, conforme os sistemas de coordenada, mas estes serão identificados nas explicações conforme necessário.

Do *pipeline 2D*, podemos organizar como os Sistemas de Referência (SR) foram organizados da seguinte forma:

1. **SR do Objeto (SRO):** todas as entidades são polígonos quadrados, e seus tamanhos são pré-definidos na inicialização do programa de forma estática. Dessa forma, o SRO é extremamente simples e só define

a posição das arestas deste polígono utilizando esse tamanho pré-definido. O desenho destes polígonos é abordado em mais detalhes na subseção sobre o *callback* `mainDraw`.

2. **SR do Universo (SRU):** as coordenadas lidas e armazenadas, conforme descrito na subseção sobre o carregamento de dados, tem uma relação 1:1 com as coordenadas do SRU. Dessa forma, o posicionamento dos objetos trivialmente utiliza estas coordenadas, armazenadas no objeto da Entidade.
3. **SR da Window (SRW):** a janela de recorte é definida a partir dos maiores e menores limites do SRU em que existem posições de alguma entidade qualquer. Isso significa que, ao ler as posições possíveis no carregamento de dados, o maior e menor valor de *x*, assim como de *y*, eram utilizados para posicionar a janela de recorte no SRU.
4. **SR da Viewport (SRV) e do Display (SRD):** o tamanho da *viewport* é estaticamente definido, e do *display* também como sendo os mesmos valores que a *viewport*. Isso é proposital para não esticar a imagem de alguma forma.

Já o *pipeline* deste trabalho pode ser dividido nas etapas enumeradas a seguir:

1. Carregamento de dados
2. Registro do *callback* de renderização
3. Registro do *callback* de animação
4. Registro do *callback* de interação com teclado
5. Definição da *viewport* e da projeção ortográfica de recorte
6. Início do *loop principal*

3.1. Carregamento de Dados

O arquivo de trajetórias é lido na função `initializeEntities` a partir de um

arquivo recebido por parâmetro da aplicação. A primeira linha é ignorada, e as subsequentes representam, cada uma, uma entidade. A definição de cada pode ser dada da seguinte forma:

$$F_M (X_0, Y_0, F_0) (X_1, Y_1, F_1) \dots (X_n, Y_n, F_n)$$

Onde F_M é a quantidade máxima de *frames* em que a entidade tem posição, e cada X_n, Y_n, F_n subsequentes representam uma posição em um determinado *frame*. Estas coordenadas já se encontram definidas em relação ao SRU. Uma entidade adicional (*mainEntity*) é criada no centro da cena, e uma flag interna do objeto a define como controlável.

O valor máximo e mínimo dentre todas as coordenadas é lido e registrado para definir os limites máximos da janela de recorte (definidos em *gluOrtho2D*), e definem assim o SRW. Dessa forma, as coordenadas lidas no arquivo tem uma relação 1:1 com o SRW, simplificando a implementação.

As entidades são instâncias da classe *Entity*, que possuem um vetor *positions* de instâncias de *EntityPosition*.

3.2. Registro do *callback* de renderização

Nesta etapa, registra-se em *glutDisplayFunc* o *callback* principal de renderização dos objetos no SRU (função *mainDraw*). Como este projeto não possui ações de zoom ou movimentação de câmera, não há necessidade de configurar a matriz de projeção (*GL_PROJECTION*) aqui. Define-se então apenas a matriz de modelo e visualização (*GL_MODELVIEW*), configurando o fundo na cor preta e iterando as entidades para desenhá-las. Ao final, usa-se *glutSwapBuffers* para realizar a troca

entre o *frame* desenhado a recém e o *frame* sendo exibido. A utilização de *buffer* duplo impede a ocorrência de *flickering*.

O desenho de cada entidade é responsabilidade da função *drawEntity*, que utiliza as propriedades e métodos da instância de *Entity* recebida por parâmetro para desenhá-la. Utiliza-se a propriedade do tipo *Color* para definir a cor do objeto e obtém-se a posição do mesmo através do método *getCurrentPosition*. A posição é então utilizada com *glTranslatef* para posicionar o objeto no SRU. Detalhes sobre o cálculo da posição atual da entidade são abordados na subseção "Posicionamento da Entidade e Interpolação Linear".

O desenho da entidade utiliza a forma convencional de desenho de primitivas do *OpenGL*, valendo-se de vértices do tamanho configurado na variável global *entities_size*, e finaliza restaurando a matriz de transformações.

3.3. Registro do *callback* de animação

A função *animate()* é utilizada como *callback* de *glutIdleFunc*, e fica responsável por atualizar o estado da simulação a 60 *frames* por segundo (FPS). Este *callback* calcula um Δt , a diferença temporal entre cada *frame* a ser renderizado, só permitindo sua execução a cada 1/60 segundos e garantindo a atualização do estado apenas 60 vezes por segundo.

A atualização principal de estado neste programa envolve iterar cada entidade, atualizar um contador interno para cada uma através do método *Entity::updateCurrentFrame*, e calcular as colisões da cena atual por um objeto *CollisionChecker*. Depois, para cada entidade em colisão, calcula-se uma nova cor para o par de entidades em colisão.

O contador interno nada mais é do

que uma indicação do *frame* atual em que a entidade se encontra, dentre as posições de sua lista de posições. A diferença crucial se encontra em como a posição é processada: cada incremento de *frame*, na realidade, incrementa este contador em 0.1. Isso cria 8 *frames* adicionais entre cada posição original da entidade, cuja posição atual deve ser calculada por interpolação linear das posições vizinhas (e.g., contador atual em 5.2 teria como vizinhos os *frames* 5 e 6).

Além disso, para evitar entidades paradas ou saltos de posição muito bruscos, uma *flag* interna da entidade define se o contador incrementa ou decrementa. Isso é utilizado para, ao finalizar a quantidade de posições que a entidade possui, ela começar a "andar para trás", isto é, decrementa o seu contador de *frame* interno até o começo. Ao voltar do início, a direção habitual de incrementar o contador é retomada.

A última etapa do *callback* responsável pela animação trabalha com a colisão das entidades. Após atualizar a posição de cada entidade, um novo objeto do tipo `CollisionChecker` é alimentado com estas posições. Este objeto processa pares entre cada entidade e calcula suas colisões. Mais detalhes sobre o cálculo de colisão encontra-se na subseção *Deteção de Colisão*.

Essas colisões então são retornadas para que o *callback* de animação realize alguma ação com as entidades que colidiram. No caso deste trabalho, optou-se por criar uma cor aleatória e atribuí-la para ambas entidades envolvidas na colisão, e com isso o processamento do estado da aplicação finaliza para mais um *frame*.

A atribuição de uma nova cor ocorre a cada colisão detectada. Isso significa que se uma entidade permanecer sobreposta em outra em mais de um *frame*, a cada *frame* se atribuirá uma nova cor para os envolvidos. Isso gera um efeito "piscante" na entidade e,

apesar de não ser intencional, optou-se por manter esse resultado por se tratar de um efeito interessante para demonstrar a colisão contínua, assemelhando-se a um efeito de dano atribuído, comum em jogos eletrônicos.

3.4. Registro do *callback* de interação com teclado

As interações com o teclado são definidas pelos *callbacks* de `glutKeyboardFunc` e `glutSpecialFunc`. O primeiro define apenas uma ação simples de encerramento do programa através da tecla ESC, e o segundo define a interação que movimenta a entidade principal, ou *avatar*. Este movimento utiliza as teclas direcionais do teclado, mapeadas no código pelas constantes do *OpenGL* `GLUT_KEY_LEFT/RIGHT/UP/DOWN`, equivalentes às teclas respectivas no teclado.

A utilização destas teclas incrementa ou decrementa uma propriedade `overridePosition` da entidade controlável em cada eixo, em uma quantidade pré-definida, igual para cada direção. Além disso, não é possível complementar duas direções simultaneamente (como para cima e para direita simultaneamente, por exemplo).

Essa propriedade é inicializada com o centro do SRW ($\frac{X_{Window}}{2}, \frac{Y_{Window}}{2}$), modifica-se com as teclas direcionais, e intercede diretamente no método `Entity::getCurrentPosition` sobreescrevendo a posição retornada pelo método com si. Isso se converte com a movimentação da entidade *avatar* pelo SRU.

3.5. Definição da *viewport*, da projeção ortográfica de recorte, e início do loop principal

Na última etapa do *pipeline*, definimos o tamanho da *viewport* em 800 por 800 *pixels*,

e o recorte da janela utilizando os valores máximos e mínimos de x e y das coordenadas lidas no arquivo no começo do processo. Como já mencionado, não há necessidade de atualizar a janela de recorte em outros momentos por não haver interações que a alterem, como *zoom* ou movimento da camera nos eixos do SRU. O fluxo finaliza iniciando o *loop* principal da *GLUT*.

3.6. Posicionamento da Entidade e Interpolação Linear

A posição de renderização da entidade no SRU é obtida a partir da lista de posições carregadas do arquivo de entrada no início do programa. A diferença é que, para suavizar o movimento, uma interpolação é calculada entre duas posições vizinhas. Essa interpolação leva em consideração que o contador de *frames* interno da entidade é decimal, como demonstrado na subseção "Registro de *callback* de animação", e utiliza essa diferença decimal entre um *frame* e outro para realizar a interpolação.

Os frames vizinhos são encontrados através dos arredondamentos para cima e para baixo desse contador interno, e a diferença entre este contador e o vizinho anterior é utilizada como fator de interpolação t - ou seja, este fator representa o progresso fracionário entre um ponto e outro. Seja a definição da interpolação linear:

$$\frac{x - x_0}{y - y_0} = \frac{x_1 - x_0}{y_1 - y_0}$$

E a definição do fator t como o progresso de x_0 até x_1 (ou seja, a razão entre a distância percorrida de x_0 até x e a distância total entre x_0 a x_1):

$$t = \frac{x - x_0}{x_1 - x_0}$$

Podemos chegar numa equação que, dado o fator t que utilizamos de entrada

(neste caso, a diferença fracionária entre os *frames* vizinhos) isolamos y e a definição de t :

$$y - y_0 = (x - x_0) \frac{y_1 - y_0}{x_1 - x_0}$$

$$y - y_0 = \frac{x - x_0}{x_1 - x_0} (y_1 - y_0)$$

$$y - y_0 = t * (y_1 - y_0)$$

$$y = y_0 + t * (y_1 - y_0)$$

A fórmula $y = y_0 + t * (y_1 - y_0)$ pode ser reescrita em uma notação vetorial $P(t) = P_0 + t * (P_1 - P_0)$, pois pouco importa se estamos tentando encontrar a coordenada x ou y de um ponto, a fórmula é a mesma. Utilizando-a em conjunto com o fator t que já possuímos, é possível suavizar o movimento das entidades.

3.7. Detecção de Colisão

O objeto `CollisionChecker`, ao ser instanciado, cria pares de cada entidade com as outras e calcula a colisão entre cada uma. Apesar da complexidade $O(n^2)$ desta abordagem, por se tratar de um número relativamente baixo de entidades nos arquivos do *Cultural Dataset*, optou-se pela simplicidade desta implementação.

A detecção então utiliza um algoritmo simples de *Axis-Aligned Bounding Box* (AABB). Como as entidades já são polígonos primitivos quadrados, o tamanho, os vértices e arestas do próprio objeto são considerados como os limites da *bounding box*. A colisão é confirmada quando ocorre sobreposição das entidades no eixo x e y , levando em consideração o próprio tamanho das entidades (globalmente configurado através da variável `entities_size`).

4. Como Executar o Código e Resultados Obtidos

O código é compilado através da ferramenta *CMake* e executado via linha de comando,

exigindo que se informe o caminho para o arquivo de dados.

4.1. Compilação e Execução do Programa

A compilação foi definida de forma exclusiva através do *CMake*:

```
cmake -build
./cmake-build-debug -target
CG_TD1
```

O programa deve ser executado passando o arquivo de dados como argumento:

```
./CG_TD1
<caminho/para/o/Paths_D.txt>
```

Como já mencionado, para controlar o *avatar*, utilizam-se as teclas direcionais do teclado, e ESC para encerrar o programa.

4.2. Resultados Obtidos

A simulação 2D exibe as entidades (quadrados) se movendo suavemente de acordo com as trajetórias definidas no arquivo informado como argumento do programa. A **avaliação visual** confirma o funcionamento da detecção de colisão: quando quaisquer duas caixas delimitadoras se sobrepõem, o par de entidades imediatamente muda para uma nova cor aleatória. A interpolação garante que o movimento seja mais fluido do que seria se apenas os pontos de *frame* fixos fossem utilizados. Além disso, foi possível trabalhar com os conceitos de cada SR e o pipeline completo do SRO para sua instânciação no SRU, o recorte da SRW e proporcionalidades em SRV e SRD.

4.3. Descrição de Uso de LLMs

Neste projeto, ferramentas baseadas em LLMs (chamadas de ferramentas de IA generativa) se concentraram em dúvidas pontuais sobre *OpenGL*, sobre especificidades da linguagem C++, sobre LaTeX na confecção deste artigo, e por fim em revisões

gerais sobre o artigo e o vídeo criado para a avaliação.

Essas ferramentas também foram utilizadas na interpolação linear: em algumas implementações, a fórmula da interpolação calculava a posição de um ponto com base em um fator t , porém a maioria das definições colocavam a formulação a partir da igualdade entre a relação $\frac{x-x_0}{y-y_0} = \frac{x_1-x_0}{y_1-y_0}$. Para não utilizar a abordagem em t cegamente, se utilizou brevemente essas ferramentas para entender o processo algébrico de transformação de uma em outra.